Adaptive Data Quality Assessment for Machine Learning Pipelines: Predicting Model Degradation Before Training

Jeff G Wilson
Department of CSE AI ML & DS
Marwadi University
Rajkot, Gujarat, India
jeff.wilso143639@marwadiuniversity.ac.in

Kishan Makadiya
Department of Computer Engineering
Marwadi University
Rajkot, Gujarat, India
kishan.makadiya@marwadieducation.edu.in

***Abstract*—Wrong labels, empty cells, repeated rows and lopsided classes lower the accuracy of machine learning models, but we usually learn how much only after training. We ask whether this loss can be estimated from the training data alone, before the target model is trained. We damaged 48 OpenML-CC18 classification datasets with several defect types and measured the loss of up to five models on clean test data. Sixteen meta-features describe each damaged training set, and a random forest meta-model predicts the loss from seven of them. In leave-one-dataset-out validation it had a lower error than a naive baseline for logistic regression, random forest and XGBoost (0.017, 0.023 and 0.030 against 0.019, 0.027 and 0.035). A test planned in advance on 27 unseen datasets, repeated with ten seeds, confirmed this for label noise with random forest, XGBoost and LightGBM (23% to 26% lower error, Holm-corrected $p < 0.02$), but not for logistic regression or a neural network. Noise estimates from cleanlab alone were weaker (6% to 15%); added as meta-features they raised the gain to 30% to 35%, and with train and test datasets swapped the intervals excluded zero. A baseline told the true noise level still did better, and the method failed on instance-dependent noise. The result also held for 500-tree versions of the three tree models. The three landmarking learners take about 0.12 s, which is faster than training a random forest on every dataset but slower than logistic regression. The method is thus an early warning for random label noise in tree-based models, not a general data-quality predictor.**

***Index Terms*—data quality, meta-learning, meta-features, label noise, data-centric AI, tabular classification**

I. Introduction

A model is only as good as the data it learns from. Real training sets contain mislabelled examples, empty cells, repeated rows and classes of very different sizes, and data-centric AI argues that improving the data often pays off more than tweaking the model [1]–[3]. Cleaning is costly, so one has to decide which problem to tackle first. This paper asks: before training the target model, how much accuracy will the problems in this dataset cost?

Most existing work looks at the problem from the other side. Some studies damage data on purpose and report how far performance falls [4], [5], surveys list data quality dimensions and cleaning tools [6], [7], and a large literature on label noise asks how to find wrong labels or train models that tolerate them [8]–[11]. In every case the damage is measured or repaired after a model exists. Another line of work uses dataset descriptors to predict how well an algorithm will do [12]–[14], but it predicts performance on the data as given, not the loss a defect causes. In our literature searches (arXiv, publisher sites and recent surveys, up to October 2026) we found no work that predicts the size of this loss from properties of the training data alone.

We frame this as a meta-learning problem. For a training set that may contain defects, we compute a short vector of meta-features: simple statistics such as the share of missing cells, and the accuracies of a few very fast landmark learners [15]–[17]. A meta-model maps this vector to the expected accuracy loss of a target model on clean test data. "Before training" means before the target model is trained: the landmark learners are themselves trained, but they are tiny and use at most 1000 rows (Section IV reports the measured time). A practitioner usually does not know how noisy the labels are, so the meta-model must estimate the damage without being told the noise level. We position the method as an early warning for label noise, not as a general data-quality predictor.

We ask three questions. **RQ1:** Can the accuracy loss from data defects be predicted from meta-features before training, better than a naive guess? **RQ2:** Which meta-features are the most informative? **RQ3:** Does it work on unseen datasets, and for which defects and models? Our contributions are:

- a controlled experiment injecting defects into 48 OpenML-CC18 datasets and a meta-learning formulation with 16 meta-features;
- a pre-planned test on 27 unseen datasets, repeated with ten seeds, that backs the label-noise result for tree-based models but not for logistic regression or a neural network;
- a comparison with confident learning (cleanlab), a measurement of the time the meta-features take, further tests with other splits, and an account of where the method fails, including instance-dependent noise.

II. Related Work

Data quality and label noise. Surveys [1], [3] and DataPerf [18] argue that data deserves as much care as models; Zhou et al. [6] review quality dimensions and tools, and Côté et al. [7] examine 101 publications combining cleaning with machine learning. The closest empirical study is that of Mohammed et al. [4], who corrupt real datasets along six dimensions and train 19 algorithms; completeness and target accuracy problems hurt most and uniqueness problems hardly mattered. Maurino [5] runs a similar study for credit risk, and Zhu et al. [19] stress-test ML pipelines by searching for the corruptions that harm a pipeline most, which requires running it. For label noise, there are surveys [8]–[10], confident learning finds label errors from out-of-sample predicted probabilities [11], and for tabular data Eisenbürger et al. [20] and Morampudi et al. [21] study boosted trees under noise. These methods need a trained classifier or report the damage afterwards; we predict the cost with cheap features, and we compare against confident learning.

Meta-learning. Meta-learning learns from earlier tasks [17]. Datasets are described by meta-features, from simple counts to the accuracy of fast learners (landmarking) [15], [16], mostly to select or configure algorithms [14] or to predict accuracy [12], [13]. We aim the same ingredients at a different target: the accuracy that defects take away. We use OpenML-CC18 [22], tree ensembles remain strong on tabular data [23], [24], and we compare methods with the Wilcoxon signed-rank test [25].

III. Method

Let D be a clean dataset, split into a training part D_{tr} and a test part D_{te}, and let a defect process turn D_{tr} into a damaged set $\tilde{D}_{tr}$. For a model family M, the degradation is $\Delta_M = \mathrm{acc}_M(D_{tr}) - \mathrm{acc}_M(\tilde{D}_{tr})$ on the clean test part, so a positive value means accuracy was lost. We compute a meta-feature vector $\varphi(\tilde{D}_{tr})$ and learn for each M a function f_M with $f_M(\varphi(\tilde{D}_{tr})) \approx \Delta_M$ that should work on datasets not used to learn it. Since φ uses the damaged training data only, f_M can be applied before the target model is trained.

Figure 1 gives the overall picture. In lane A we compute the meta-features from the damaged training data and pass them to the meta-model, which returns a predicted loss; this is all a practitioner would need. Lane B exists only to build and check the meta-model: we train a target model on the damaged data, test it on the clean test part, and obtain the real loss that the meta-model learns to predict.

A. Datasets and defects

We draw 48 datasets from OpenML-CC18 [22] in two rounds. Round 1 has 21 datasets from a published CC18 table (a convenience subset; one high-dimensional dataset was left out to save time). Round 2 adds 27 datasets chosen by a rule fixed before any result on them was seen: all remaining CC18 datasets with at most 10,000 rows, 100 columns and 15 classes. The OpenML IDs are listed in the code repository. Categorical columns become integer codes and missing values stay missing.

For each seed we split a dataset once into a 70% training and a 30% test part, stratified by class, and inject defects into the training part only. *Missing values:* each cell is set to missing with probability s (completely at random). *Label noise:* a fraction s of the labels is replaced by another class chosen uniformly (symmetric noise). *Duplicates:* $s \times n$ random rows are appended as copies. *Imbalance:* a fraction s of the examples of every class except the largest is removed (at least 2 are kept). Later runs add *class-dependent noise* (a fraction s of the labels moves to the next class in a fixed cyclic order, so for two classes it equals symmetric noise) and *instance-dependent noise* (a random forest estimates how uncertain each example is from out-of-bag probabilities; examples are flipped with probability proportional to their uncertainty, to the most likely other class). Severities are s = 0.05, 0.10, 0.20, 0.40, and all four basic defects are also combined at 0.10 and 0.20.

B. Target models, meta-features and meta-models

The target models are logistic regression, a 100-tree random forest and XGBoost, and later LightGBM and a small neural network (MLP), all with library defaults [26], [27]; missing values are filled with the training median. We measure accuracy and, in later runs, balanced accuracy and macro-F1 on the clean test part.

The 16 meta-features are computed on the damaged training part: *basic* (rows, columns, classes, rows per column), *data quality* (missing_rate, duplicate_rate, imbalance_ratio, outlier_rate), *statistical* (mean skewness, kurtosis and absolute correlation), *information-theoretic* (class_entropy, mean_mutual_info) and *landmarking* (3-fold cross-validated accuracy of a decision stump, a depth-3 tree and naive Bayes: lm_stump, lm_tree3, lm_nb). Model-based features use a random sample of at most 1000 rows. In later runs we add four features from confident learning [11], computed with the cleanlab library on out-of-sample probabilities of a 50-tree forest: the share of examples flagged as label issues, cleanlab's estimated noise rate, the mean probability given to the given label, and the share of disagreements.

The meta-model is a random forest regressor (200 trees, at least 5 samples per leaf, half the features per split) on the "small set" of seven features: missing_rate, duplicate_rate, imbalance_ratio, class_entropy, lm_stump, lm_tree3 and lm_nb (Forest (small set)). Baselines are *Average drop* (the mean degradation seen in training), a linear rule on the three defect rates, ridge regression, and a forest on all features. Later runs add *cleanlab only* (isotonic regression from cleanlab's noise estimate or share of flagged issues to the loss), the small set plus the cleanlab features, landmarking plus cleanlab, gradient boosting, and a baseline told the true noise level.

C. Evaluation protocol

Results are averages over three seeds in the main experiment and ten seeds in the extended runs, each seed with its own split and random defect draws. We report the

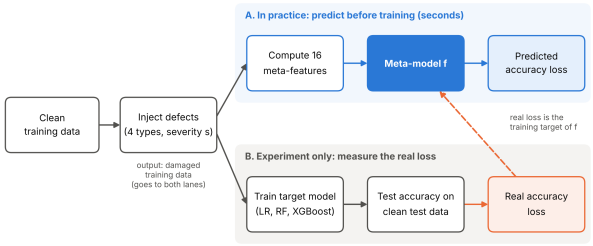


Fig. 1. Overview of the method. Lane A (blue) is the intended use: predict the accuracy loss from the damaged training data before training the target model. Lane B (grey) is the experiment: measure the real loss, which serves as training target and ground truth.

mean absolute error (MAE) of the predicted degradation, the Spearman correlation ρ, and, at the dataset level, the error reduction against Average drop with a 95% bootstrap interval (2000 resamples of datasets) and one-sided Wilcoxon signed-rank tests, Holm-corrected in the extended runs. *Analysis 1 (exploratory)* is leave-one-dataset-out validation on all 48 datasets. It is exploratory because the small set was chosen after a first look at the 21 Round-1 datasets, which led us to H1. *Analysis 2 (planned)* is the test we trust most. H1 was written down before any Round-2 dataset was run: *on the label-noise cases of unseen datasets, Forest (small set) has a lower mean error than Average drop.* We train on the 21 Round-1 datasets and test on the label-noise cases of the 27 Round-2 datasets, separately for each model, with a one-sided Wilcoxon test at $0.05/3 \approx 0.0167$. *Analysis 3 (extended runs, after the main results and in response to reviews)* adds more seeds, models, measures and noise types, cleanlab, other splits and timing; it is exploratory. Everything ran in Python on Google Colab with scikit-learn, XGBoost and OpenML [26]–[28]. Code, notebooks and result files are available at https://github.com/jeffgwilson0-png/Data-quality-degradation.

IV. Results

A. Loss by defect, and prediction on 48 datasets

Label noise is by far the most damaging defect: on average it costs 3.6, 5.3 and 7.7 percentage points (pp) for logistic regression, random forest and XGBoost, and 8, 16 and 20 pp at 40% noise. Missing values cost 1.1 to 1.5 pp, combined defects 3.1 to 5.2 pp, and class imbalance and duplicate records at most 0.4 pp. In the three-seed run, class-dependent noise cost more than symmetric noise (4.8 to 9.5 pp against 3.2 to 7.7 pp), and imbalance cost more under balanced accuracy and macro-F1 (0.9 to 2.6 pp) than under accuracy (0.3 to 0.8 pp), but still little.

In leave-one-dataset-out validation (Analysis 1), Forest (small set) had the lowest MAE for all three model families: 0.017, 0.023 and 0.030 against 0.019, 0.027 and 0.035 for Average drop, better on 37, 39 and 37 of 48 datasets (Wilcoxon $p = 0.005$, 0.001, 0.002). A forest on all features behaved much the same for random forest and XGBoost ($p = 0.005$) but not for logistic regression ($p = 0.082$), and ridge regression and the defect-rate rule gave little or no gain. With 12 comparisons the Bonferroni threshold is about 0.004, so only the random forest and XGBoost results pass clearly, and because the small set was chosen after a first look, they are exploratory. The rank correlations are only 0.40 to 0.48, and the largest losses, from label noise at high severity, are under-predicted (Fig. 2a). By defect type the gain comes from label noise with random forest and XGBoost targets (MAE 0.036 and 0.047 against 0.047 and 0.063), not for logistic regression (0.030 against 0.031); for missing values the meta-model is no better than the baseline, and for duplicates and imbalance its errors (0.008 to 0.023) are much larger than the real losses (at most 0.004): it predicts a loss that does not happen.

B. Planned test on unseen datasets

In the main experiment (three seeds), H1 was supported for random forest and XGBoost: Forest (small set) had a lower error on 23 of the 27 datasets (MAE 0.0345 against 0.0464 and 0.0479 against 0.0617, 26% and 22% lower; Wilcoxon $p = 0.0004$, below the 0.0167 threshold), but not for logistic regression (better on 15 of 27, 0.0307 against 0.0285, $p = 0.48$). The rank correlations were 0.36, 0.68 and 0.54 for logistic regression, random forest and XGBoost. A baseline that predicts the mean label-noise loss was worse than Average drop (0.0368, 0.0563, 0.0714), probably because the losses are skewed. The meta-model fails on a few small datasets (Fig. 2b): dresses-sales (350 rows) and analcatdata_dmft (557 rows) have errors of 0.14 and 0.05 for random forest against 0.01 and 0.02 for the baseline, and across datasets the error increase rises with the number of classes (Spearman 0.67 for random forest, 0.58 for XGBoost); we did not test why.

Ten seeds, four models, three measures. We repeated the test with ten seeds (the three seeds of the main run re-used, seven new), adding LightGBM, balanced accuracy and macro-F1 (Table I). For accuracy, the meta-model beat Average drop for the three tree-based models (reductions of 26.5%, 22.8% and 23.1%, better on 21, 23 and 24 of 27 datasets, Holm p of 0.018, 0.019 and 0.015) and not for logistic regression (-3.6%). For macro-F1 the gain is significant for XGBoost (16.2%) and LightGBM (16.8%) but not for random forest (8.4%, $p = 0.12$), and for balanced accuracy for none (best 11.7%, $p = 0.11$). An earlier three-seed run with a neural network (MLP) also showed no gain (1.3%, $p = 1$). The intervals still include zero in every row, so more seeds

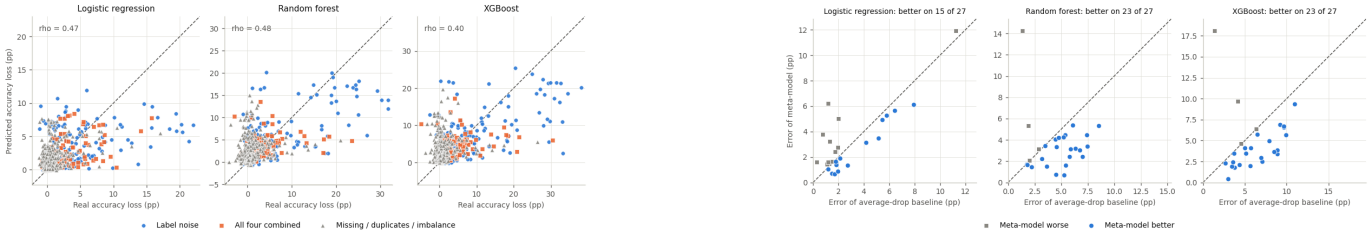


Fig. 2. (a) Predicted versus real accuracy loss of Forest (small set) in leave-one-dataset-out validation; each point is a dataset and defect setting, and ρ is the Spearman correlation. (b) Planned test on unseen datasets (label noise only); each point is a dataset, and points below the dashed line are datasets where the meta-model has a lower error than Average drop.

confirmed the direction of the gain but did not pin down its size.

C. Cleanlab and other competitors

Table II compares meta-models and baselines on the accuracy loss under label noise, with the same split. Cleanlab alone, mapping its estimated noise rate or its share of flagged issues to the loss, reduced the error by only 6% to 15% for the tree models, against 23% to 26% for Forest (small set). Adding the four cleanlab features raised the reduction to 30% to 35% (for macro-F1, 12%, 23% and 21% for random forest, XGBoost and LightGBM, against 8%, 16% and 17%), and a forest on only the landmarking and cleanlab features did as well. A forest on all 20 features and gradient boosting lay in between (26% to 30%), and ridge regression was worse than the baseline. No method that was not told the noise level beat the baseline for logistic regression. We did not test whether the differences between these meta-models are significant in this split.

D. What the meta-model offers beyond the noise level

A baseline told the true noise level beats every method that is not told it (47% to 52% for the tree models), so the meta-model mostly acts as a noise estimator. Two checks show how much more it offers. First, adding the true noise level to the small feature set raised the reduction from 47%–52% to 55%–58% (last row of Table II), so the dataset features add about 4 to 8 points beyond the noise level. Second, we asked whether the meta-model can rank datasets at a fixed noise level (0.1, 0.2 and 0.4). The rank correlation between predicted and real loss within a noise level, averaged over levels, was 0.42, 0.21 and 0.22 for random forest, XGBoost and LightGBM (0.02 for logistic regression), and zero or negative for cleanlab alone. If only the 25% of datasets with the highest predicted loss could be audited, they would hold 34%, 30% and 30% of the total loss (random choice 25%, perfect ranking 41% to 47%), but the bootstrap intervals for the difference from random choice include zero (for example -2 to $+18$ points for random forest). The meta-model thus carries some dataset-specific signal, too weak to be established here.

E. More evidence with other splits

Table III uses three further tests. In leave-one-dataset-out validation over all damaged settings (including class-dependent noise), Forest (small set) reduced the error by 22% for the tree models (Holm $p \leq 0.0012$); with the cleanlab features the reduction was 34% to 35%, and the lower interval bounds were above zero (7% to 9%). Leaving out whole clusters of similar datasets (six clusters by size, number of features and classes) gave 17% to 20% and 27% to 31%. When we swapped the roles and trained on the 27 Round-2 datasets to test on label noise in the 21 Round-1 datasets, the reductions were 26% to 32% for the tree models with intervals that exclude zero, and 42% to 46% with the cleanlab features; for logistic regression the gains were not significant after correction ($p = 0.084$ and 0.055). The cleanlab features were added in response to the reviews and are one of many variants compared, so these differences remain exploratory.

F. Other noise types

A meta-model trained without any class-dependent noise reduced the accuracy error on class-dependent noise in the 27 new datasets by 24.1% (random forest, Holm $p = 0.032$), 18.2% (XGBoost) and 19.4% (LightGBM), but not for logistic regression or the MLP. On the 13 multi-class datasets, where class-dependent and symmetric noise really differ, the reductions shrink to 5.8%, 6.1% and 5.5% (intervals include zero), so most of the apparent transfer comes from the 14 binary datasets, where the two noise types coincide. For instance-dependent noise (three seeds, 27 new datasets), the real mean accuracy loss was 3.6 to 8.3 pp, but Forest (small set) did not beat Average drop for any model (reductions of -5.1%, -0.1%, -3.3% and -5.0% for logistic regression, random forest, XGBoost and LightGBM; Holm $p \geq 0.47$). Adding the cleanlab features made it worse (4% to 11% worse than Average drop), and cleanlab alone was worst (13% to 20% worse), whereas the baseline told the noise level cut the error by 42% to 47%. The method therefore detects random label noise, not noise that depends on the features, which is the more realistic kind; we did not test why.

G. Larger target models

To test whether the result depends on model size, we also trained 500-tree versions of random forest, XGBoost (depth 8, learning rate 0.05) and LightGBM (63 leaves, learning rate 0.05) on the same damaged training sets (three seeds), next to the standard-size models, and repeated the planned test (Table IV). For accuracy, the larger models behave like

TABLE I

PLANNED TEST WITH TEN SEEDS (LABEL-NOISE CASES OF THE 27 NEW DATASETS, META-MODEL TRAINED ON ROUND 1; BALANCED ACCURACY OMITTED, SEE TEXT). MAE OF AVERAGE DROP, OF THE META-MODEL AND OF A BASELINE TOLD THE TRUE NOISE LEVEL; ERROR REDUCTION OF THE META-MODEL WITH 95% BOOTSTRAP INTERVAL; DATASETS WHERE IT WINS; HOLM-CORRECTED p.

Measure	Model	Avg. drop	Meta-model	Told noise	Reduction (%)	95% CI	Better	Holm p
Accuracy	logreg	0.0288	0.0298	0.0292	−3.6	[−26.6, 12.6]	15/27	1.000
	rf	0.0461	0.0339	0.0246	26.5	[−4.1, 46.6]	21/27	0.018
	xgb	0.0608	0.0470	0.0295	22.8	[−6.3, 42.0]	23/27	0.019
	lgbm	0.0545	0.0419	0.0263	23.1	[−5.8, 42.6]	24/27	0.015
Macro-F1	logreg	0.0555	0.0578	0.0545	−4.2	[−23.8, 8.2]	17/27	1.000
	rf	0.0482	0.0442	0.0323	8.4	[−19.4, 27.3]	20/27	0.123
	xgb	0.0686	0.0575	0.0395	16.2	[−7.2, 32.8]	23/27	0.019
	lgbm	0.0622	0.0517	0.0344	16.8	[−8.3, 34.1]	23/27	0.018

TABLE II

ERROR REDUCTION (%) AGAINST AVERAGE DROP IN THE PLANNED TEST (ACCURACY LOSS, LABEL NOISE, 27 NEW DATASETS, TEN SEEDS). THE LAST TWO ROWS USE THE TRUE NOISE LEVEL, WHICH A PRACTITIONER WOULD NOT KNOW.

Method	logreg	rf	xgb	lgbm
Cleanlab: share of issues only	−23.5	10.7	8.7	9.8
Cleanlab: noise rate only	−28.2	6.3	14.6	10.0
Ridge (all 20 features)	−80.2	−38.0	−26.4	−32.4
Gradient boosting (all 20)	−12.3	29.6	26.2	25.5
Forest (all 20)	−8.2	29.4	28.9	27.5
Forest (small set)	−3.6	26.5	22.8	23.1
Forest (small set + cleanlab)	−10.3	35.2	32.1	29.8
Forest (landmark + cleanlab)	−16.4	34.7	32.0	29.2
Told the noise level	−1.3	46.6	51.4	51.7
Forest (small set) + noise level	2.5	54.8	57.5	56.0

the standard ones: the meta-model cut the error by 25.7%, 23.5% and 22.9% (better on 21, 23 and 24 of 27 datasets, Holm $p \leq 0.018$), although the intervals for XGBoost and LightGBM include zero, as they do for the standard models. With the cleanlab features the reductions were 27.7% to 32.9%, and a baseline told the noise level still did better (43.8% to 46.5%). For macro-F1 the gain held for XGBoost and LightGBM (14.9% and 14.2%, Holm $p = 0.023$) but not for random forest (7.1%, $p = 0.14$). A meta-model trained on the losses of the *standard-size* models predicted those of the larger models almost as well (21.7% to 24.7%, Holm $p < 0.02$), so it need not be retrained for each model size. The larger random forest and XGBoost lost slightly less accuracy to label noise than the standard ones (at 40% noise, 14.8 against 15.6 pp and 19.4 against 20.4 pp), and the larger LightGBM lost more (20.4 against 18.8 pp); these paired differences at 20% and 40% noise have uncorrected $p < 0.01$ over the 48 datasets. One fit of a larger model took a median of 1.9, 1.4 and 1.0 s, so computing all 20 meta-features (0.62 s) was faster than training on 100%, 72% and 62% of the dataset-seed pairs, and the landmarking part (0.12 s) on all of them; the feature times come from the earlier run.

H. Cost of the meta-features

Median times on a Colab CPU over 48 datasets and ten seeds: the three landmark learners take 0.12 s, the 16 meta-features 0.13 s and the cleanlab features another 0.49 s (0.61 s together). One training run on the clean training set takes 0.02 s for logistic regression, 0.41 s for random forest, 0.14 s for XGBoost and 0.16 s for LightGBM. The landmarking part is faster than one random forest fit on every dataset, than XGBoost on 64% and than LightGBM on 67% of them, but slower than logistic regression (faster on 17%); with the cleanlab features it is faster than training on only 6% to 28% of the datasets. The "before training" claim therefore means before training the target model: the landmark learners are themselves trained, but they are tiny and use at most 1000 rows, so their cost is bounded by design. Most datasets here are small, so a single fit is already fast, and we did not measure larger datasets. The features are computed once for all models and cleaning options, and training alone cannot show the accuracy loss, which needs the clean labels.

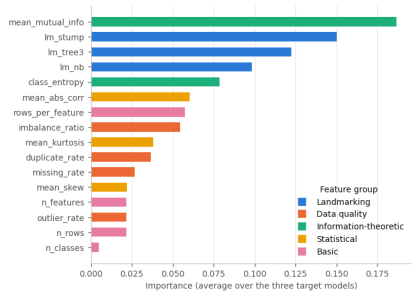


Fig. 3. Importance of the 16 meta-features in the random forest meta-model, averaged over the three target models. Colours show the feature group.

I. Which meta-features matter?

Figure 3 shows the importance of the 16 meta-features in a forest that uses all of them. The four most important are mean mutual information and the three landmarking features, followed by class entropy; together they make up a little over half of the total importance, whereas the measured duplicate and missing rates rank low. In a three-seed ablation, removing the landmarking features cut the reduction for random forest, XGBoost and LightGBM from 25.5%, 22.3% and 23.1% to 9.1%, 7.6% and 8.1%, and landmarking features alone gave 20.0%, 17.2% and 17.2%, so they carry most of the signal.

TABLE III

FURTHER TESTS, ACCURACY LOSS, TEN SEEDS: ERROR REDUCTION (%) AGAINST AVERAGE DROP WITH 95% BOOTSTRAP INTERVAL, DATASETS WHERE THE META-MODEL WINS, AND HOLM-CORRECTED p. LOO: LEAVE-ONE-DATASET-OUT OVER ALL DAMAGED SETTINGS (48 DATASETS); CLUSTER: LEAVE-ONE-CLUSTER-OUT; REVERSED: TRAINED ON THE 27 ROUND-2 DATASETS, TESTED ON LABEL NOISE IN THE 21 ROUND-1 DATASETS.

Test	Model	Forest (small set)				Forest (small set + cleanlab)			
		Red. (%)	95% CI	Wins	p	Red. (%)	95% CI	Wins	p
LOO	logreg	10.2	$[-13.2, 28.6]$	36/48	.004	15.2	$[-10.7, 34.4]$	40/48	$<.001$
	rf	21.6	$[-5.5, 41.7]$	38/48	.001	34.7	$[8.2, 54.1]$	42/48	$<.001$
	xgb	21.4	$[-3.9, 40.3]$	38/48	.001	34.5	$[9.1, 53.3]$	40/48	$<.001$
	lgbm	21.5	$[-5.0, 41.2]$	37/48	.001	33.5	$[6.8, 52.9]$	40/48	$<.001$
Cluster	logreg	6.4	$[-16.6, 24.3]$	37/48	.014	6.7	$[-19.0, 27.1]$	41/48	.004
	rf	16.5	$[-9.9, 36.4]$	38/48	.004	27.6	$[-1.2, 48.3]$	40/48	.001
	xgb	19.5	$[-4.3, 37.5]$	38/48	.006	30.8	$[3.4, 51.1]$	41/48	.001
	lgbm	18.1	$[-6.9, 37.4]$	38/48	.006	27.2	$[-1.5, 48.5]$	41/48	.001
Reversed	logreg	11.3	$[-11.2, 25.6]$	14/21	.084	26.5	$[-0.8, 43.2]$	16/21	.055
	rf	25.7	$[7.5, 38.1]$	16/21	.019	42.3	$[25.5, 54.3]$	17/21	.004
	xgb	32.2	$[19.2, 43.5]$	17/21	.003	46.1	$[32.6, 57.2]$	20/21	.001
	lgbm	30.5	$[15.0, 43.0]$	17/21	.006	45.9	$[32.0, 57.1]$	20/21	.001

TABLE IV

PLANNED TEST FOR ACCURACY WITH STANDARD AND LARGER MODELS (THREE SEEDS, 27 NEW DATASETS). ERROR REDUCTION (%) AGAINST AVERAGE DROP WITH 95% BOOTSTRAP INTERVAL.

Model	Size	Meta-model	+ cleanlab	Told noise
rf	standard	25.6 $[-4.2, 44.9]$	31.4	44.4
	larger	25.7 $[0.8, 43.5]$	32.9	43.8
xgb	standard	22.3 $[-6.0, 40.5]$	27.2	46.2
	larger	23.5 $[-7.3, 43.8]$	27.7	46.5
lgbm	standard	23.1 $[-8.3, 42.5]$	28.7	47.6
	larger	22.9 $[-5.2, 41.2]$	28.0	46.4

V. DISCUSSION

The clearest result is the planned test. A meta-model trained on only 21 datasets predicted the label-noise loss on 27 unseen datasets better than the baseline for random forest, XGBoost and (in the ten-seed repeat) LightGBM, with 22% to 26% lower error, and swapping the training and test datasets gives intervals that exclude zero (Table III). Still, the intervals of the planned test include zero, the rank correlations are only 0.5 to 0.7 and the largest losses are under-predicted. We therefore see the tool as a way to triage, not to measure: a high predicted loss is a reason to check the labels before tuning, but it does not replace running the experiment.

Beyond the noise level. A baseline told the true noise level does better, so the meta-model is mainly a noise estimator that needs no label audit. That is still useful, since a practitioner does not know the noise level beforehand. It also carries a little dataset-specific information (the dataset features add 4 to 8 points to the true noise level, and the within-level rank correlation is 0.2 to 0.4), but the gain in an audit of the top 25% is not established. Cleanlab's noise estimate alone is weaker than the landmarking features, yet complementary: together they gave the best predictor that is not told the noise level, at the price of 0.5 s of extra computing time. The meta-model relies mainly on landmarking features, which plausibly act as label-noise detectors because wrong labels weaken the accuracy of cheap landmark learners.

Where it does not work. Logistic regression and the MLP are the exceptions: their average label-noise losses are smaller (3.6 and 3.2 pp against 5.3 to 7.7 pp for tree models) and the meta-model gave no gain, even though it had for logistic regression in the first round, which shows why the planned test was worth the effort. For tree models the picture fits reports that boosted trees are sensitive to label noise [20], [21]. The method also fails on instance-dependent noise, transfers weakly to class-dependent noise on multi-class data, and gives no useful prediction for missing values, duplicates or imbalance; like Mohammed et al. [4], we find target-accuracy problems severe and uniqueness problems minor. It should be read as an early warning for random label noise in tree-based models, not as a comprehensive data-quality predictor.

Relevance to autonomous and intelligent systems. Automated ML pipelines retrain on incoming data that nobody reads. A cheap, model-agnostic estimate of how much label noise could cost lets such a system decide whether to retrain, to request relabelling or to raise an alert before spending compute on training. The use is limited by the scope above: it assumes random-like label noise and needs a meta-model trained on many datasets with known damage.

VI. LIMITATIONS

The defects are synthetic: missing values are completely at random, and label noise is symmetric, follows a fixed cyclic pattern or depends on a random forest's uncertainty. We did not test real-world label noise, and the method fails on the instance-dependent noise that is closest to it. We study tabular classification on one benchmark suite, with four to five model families at default hyperparameters (larger versions only for the three tree models, up to 500 trees, with three seeds), and datasets may have quality problems of their own [29]. The Round-1 datasets are a convenience subset, and the small feature set and the breakdown by defect type were chosen after seeing first-round results, so the 48-dataset p-values are

exploratory. The planned test covers one hypothesis for one defect type and fails for logistic regression and the MLP. The extended run, the cleanlab variants and the noise-level analyses were designed after seeing the main results, compare many variants on 27 datasets, and have bootstrap intervals that include zero. The gains for balanced accuracy and macro-F1 and for class-dependent noise on multi-class data are weak, the timing comes from mostly small datasets on one machine, and the class-dependent and instance-dependent results use three seeds.

VII. Conclusion and Future Work

We asked whether the accuracy lost to data defects can be predicted before the target model is trained. Across 48 datasets, a small random forest meta-model on seven meta-features had a lower error than a naive baseline, mostly for label noise. A planned test on 27 unseen datasets confirmed this for random forest and XGBoost, and a ten-seed repeat added LightGBM (23% to 26% lower error, Holm $p < 0.02$), but not for logistic regression or a neural network. Confident-learning estimates alone were weaker but improved the meta-model as features (30% to 35%). A baseline told the true noise level does better, the method fails on instance-dependent noise, and the gains are weaker for balanced accuracy and macro-F1, so we regard it as an early warning for random label noise in tree-based models. Future work should test real-world label noise, structured noise that depends on the features, deep networks and more datasets for training the meta-model, and include more model families such as TabPFN [30].

References

[1] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, Z. Jiang, S. Zhong, and X. Hu, "Data-centric artificial intelligence: A survey," *ACM Computing Surveys*, vol. 57, no. 5, p. Article 129, 2025.

[2] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, and X. Hu, "Data-centric AI: Perspectives and challenges," in *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM)*, 2023, pp. 945–948.

[3] S. E. Whang, Y. Roh, H. Song, and J.-G. Lee, "Data collection and quality challenges in deep learning: A data-centric AI perspective," *arXiv preprint arXiv:2112.06409*, 2021.

[4] S. Mohammed, L. Budach, M. Feuerpfeil, N. Ihde, A. Nathansen, N. Noack, H. Patzlaff, F. Naumann, and H. Harmouch, "The effects of data quality on machine learning performance on tabular data," *Information Systems*, p. Article 102549, 2025.

[5] A. Maurino, "How data quality affects machine learning models for credit risk assessment," *arXiv preprint arXiv:2511.10964*, 2025.

[6] Y. Zhou, F. Tu, K. Sha, J. Ding, and H. Chen, "A survey on data quality dimensions and tools for machine learning," in *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, 2024, arXiv:2406.19614.

[7] P.-O. Côté, A. Nikanjam, N. Ahmed, D. Humeniuk, and F. Khomh, "Data cleaning and machine learning: A systematic literature review," *arXiv preprint arXiv:2310.01765*, 2023.

[8] B. Frénay and M. Verleysen, "Classification in the presence of label noise: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 25, no. 5, pp. 845–869, 2014.

[9] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, "Learning from noisy labels with deep neural networks: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 11, pp. 8135–8153, 2023.

[10] M. Li and C. Zhu, "Noisy label processing for classification: A survey," *arXiv preprint arXiv:2404.04159*, 2024.

[11] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021.

[12] S. Uddin and H. Lu, "Dataset meta-level and statistical features affect machine learning performance," *Scientific Reports*, vol. 14, p. Article 1670, 2024.

[13] Z. Nabizadeh Shahre Babak, F. Koohestani, N. Karimi, S. Shirani, and S. Samavi, "Meta-learning for classifier selection in image datasets: A feature-driven framework for accuracy prediction," *arXiv preprint arXiv:2609.11041*, 2026.

[14] M. Garouani, "An experimental survey and perspective view on meta-learning for automated algorithms selection and parametrization," *arXiv preprint arXiv:2504.06207*, 2025.

[15] B. Pfahringer, H. Bensusan, and C. Giraud-Carrier, "Meta-learning by landmarking various learning algorithms," in *Proceedings of the 17th International Conference on Machine Learning (ICML)*, 2000.

[16] A. Rivolli, L. P. F. Garcia, C. Soares, J. Vanschoren, and A. C. P. L. F. de Carvalho, "Meta-features for meta-learning," *Knowledge-Based Systems*, vol. 240, 2022.

[17] J. Vanschoren, "Meta-learning: A survey," *arXiv preprint arXiv:1810.03548*, 2018.

[18] M. Mazumder, C. Banbury, X. Yao, B. Karlaš *et al.*, "DataPerf: Benchmarks for data-centric AI development," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[19] J. Zhu, G. Xu, F. Lorenzi, B. Glavic, and B. Salimi, "Stress-testing ML pipelines with adversarial data corruption," *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 4668–4681, 2025.

[20] A. Eisenbürger, D. Otten, A. Hudde, and F. Hopfgartner, "Training gradient boosted decision trees on tabular data containing label noise for classification tasks," *arXiv preprint arXiv:2409.08647*, 2025.

[21] A. Morampudi, P. Padi, P. K. D. Kakitapelli, and R. K. Surapani, "Noise-dependent robustness of XGBoost, LightGBM, and CatBoost," *Algorithms*, vol. 19, no. 9, p. Article 730, 2026.

[22] B. Bischl, G. Casalicchio, M. Feurer, F. Hutter, M. Lang, R. G. Mantovani, J. N. van Rijn, and J. Vanschoren, "OpenML benchmarking suites," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021, arXiv:1708.03731.

[23] D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, B. Feuer, C. Hegde, G. Ramakrishnan, M. Goldblum, and C. White, "When do neural nets outperform boosted trees on tabular data?" *arXiv preprint arXiv:2305.02997*, 2023.

[24] H.-J. Ye, S.-Y. Liu, H.-R. Cai, Q.-L. Zhou, and D.-C. Zhan, "A closer look at deep learning methods on tabular datasets," *arXiv preprint arXiv:2407.00956*, 2024.

[25] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[26] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[27] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[28] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *SIGKDD Explorations*, vol. 15, no. 2, pp. 49–60, 2013.

[29] A. Tschalzev, L. Purucker, S. Lüdtke, F. Hutter, C. Bartelt, and H. Stuckenschmidt, "Unreflected use of tabular data repositories can undermine research quality," *arXiv preprint arXiv:2503.09159*, 2025.

[30] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, no. 8045, pp. 319–326, 2025.